

Linux Basics Every DevOps Engineer Should Know

Don't memorize it. Understand why they exist.

Step 1: Create Day 2 Project workspace with Create folders

Commands:

```
$ mkdir notes logs scripts practice
$ ls
```

Notes:

As we know, **mkdir** is used to *create directories*. Here, we *first create* the **project** folder, and then we create the **required directories** inside the project folder.

Screenshot:

```
Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project (main)
$ mkdir -p notes logs scripts practice

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project (main)
$ ls
logs/  notes/  practice/  scripts/
```

Step 2: Create notes files

Commands:

```
$ cd notes
$ touch linux_day2.txt permissions.txt users.txt
```

Notes:

After creating the required directories, we use the **cd** command to change to the **notes** directory. Then, we create the required files using the touch command. This entire lab is performed using the Command Line Interface (CLI), without relying on any Graphical User Interface (GUI).

Screenshot:

```
Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project (main)
$ cd notes/

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/notes (main)
$ touch linux_day2.txt permission.txt users.txt
```

Step 3: To check permission

Commands:

```
$ ls -l
```

Notes:

To check file permissions, we use the **ls -l** command in the (CLI). Just as we checked file permissions through the Security tab in the (GUI), the CLI displays the read (r), write (w), and execute (x) permissions for each file and directory.

Screenshot:

```
Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/notes (main)
$ ls -l
total 3
-rw-r--r-- 1 Sai 197121 231 Jul 11 12:06 linux_day2.txt
-rw-r--r-- 1 Sai 197121 74 Jul 11 12:06 permission.txt
-rw-r--r-- 1 Sai 197121 78 Jul 11 12:06 users.txt
```

Step 4: Add content

Commands:

1. >

```
$ echo "Linux Day 2 Notes" > linux_day2.txt
```

2. >>

```
$ echo "cat shows file content" >> linux_day2.txt
```

3. \

```
$ echo "head shows first lines" >> linux_day2.txt \
```

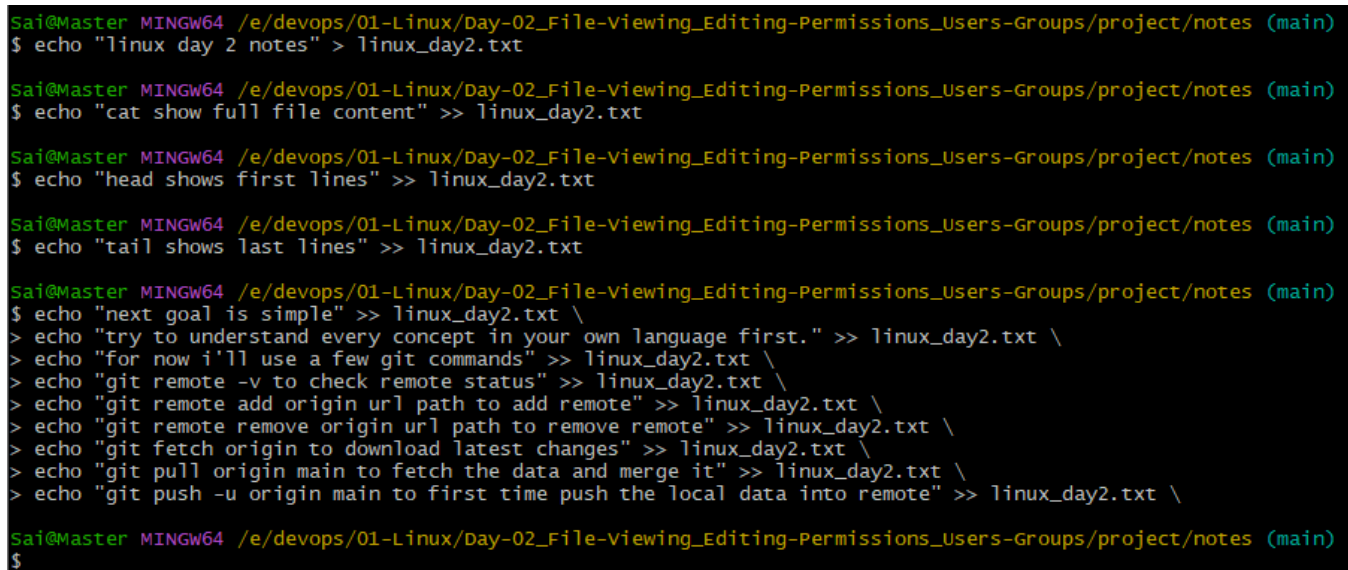
```
\ echo "less shows last line" >> linux_day2.txt \
```

Notes:

> vs >> vs \

Redirection operators (> and >>) are widely used for logging, generating configuration files, and automating tasks. The backslash (\) is commonly used in shell scripting to improve the readability of long commands and to escape special characters.

Screenshot:



```
Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/notes (main)
$ echo "linux day 2 notes" > linux_day2.txt

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/notes (main)
$ echo "cat show full file content" >> linux_day2.txt

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/notes (main)
$ echo "head shows first lines" >> linux_day2.txt

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/notes (main)
$ echo "tail shows last lines" >> linux_day2.txt

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/notes (main)
$ echo "next goal is simple" >> linux_day2.txt \
> echo "try to understand every concept in your own language first." >> linux_day2.txt \
> echo "for now i'll use a few git commands" >> linux_day2.txt \
> echo "git remote -v to check remote status" >> linux_day2.txt \
> echo "git remote add origin url path to add remote" >> linux_day2.txt \
> echo "git remote remove origin url path to remove remote" >> linux_day2.txt \
> echo "git fetch origin to download latest changes" >> linux_day2.txt \
> echo "git pull origin main to fetch the data and merge it" >> linux_day2.txt \
> echo "git push -u origin main to first time push the local data into remote" >> linux_day2.txt \

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/notes (main)
$
```

Step 5: Practice head/tail

Commands:

```
$ cat linux_day2.txt
$ head linux_day2.txt
$ tail linux_day2.txt
$ head -n 2 linux_day2.txt
$ tail -n 2 linux_day2.txt
```

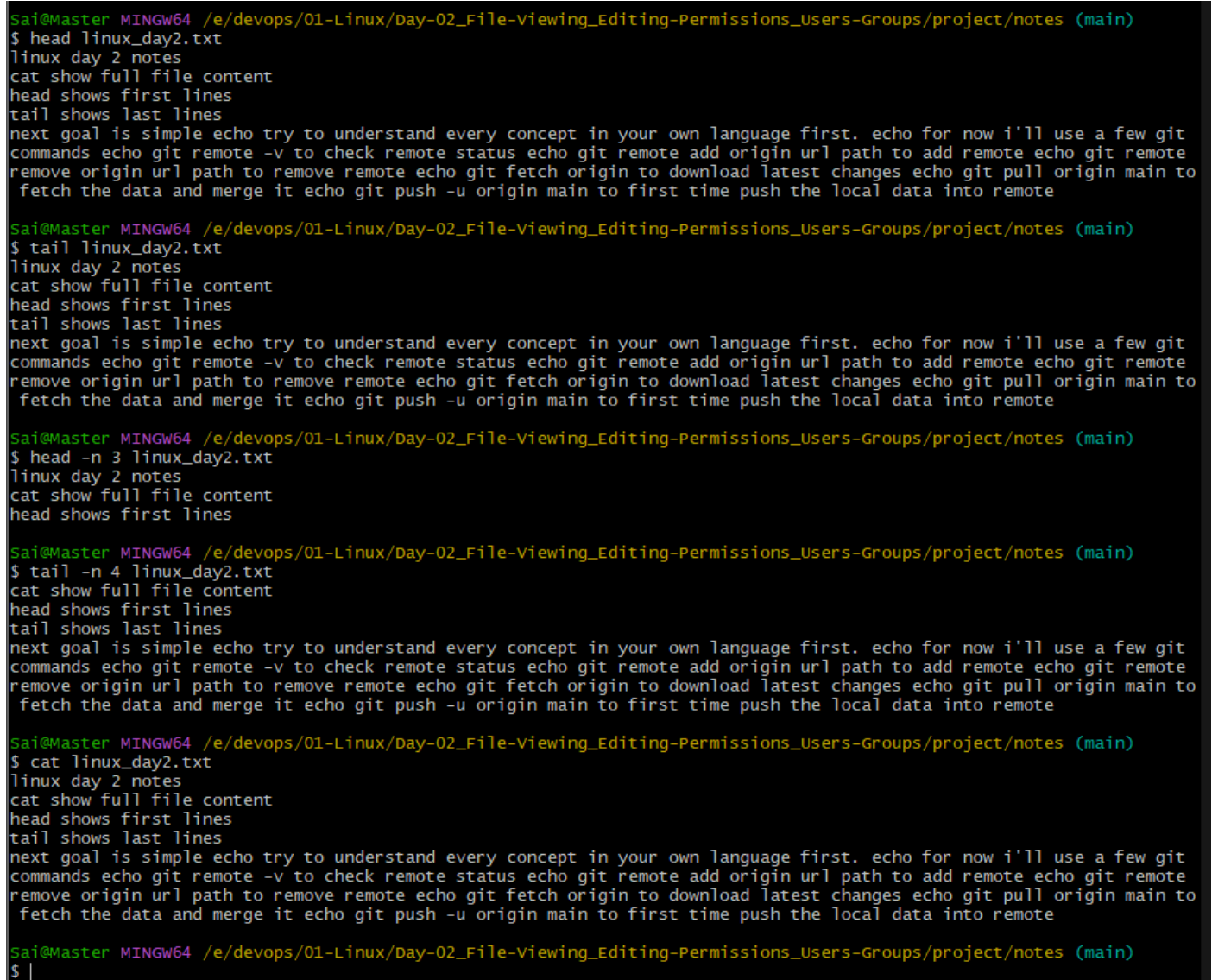
Notes:

The **cat** command is used to display the entire contents of a file, making it useful for reading small text files.

The **head** command displays the first 10 lines of a file by default, while the **tail** command displays the last 10 lines by default.

Both head and tail also support the **-n** option to display a specific number of lines, making them useful for quickly viewing the beginning or end of large files.

Screenshot:



```
Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/notes (main)
$ head linux_day2.txt
linux day 2 notes
cat show full file content
head shows first lines
tail shows last lines
next goal is simple echo try to understand every concept in your own language first. echo for now i'll use a few git
commands echo git remote -v to check remote status echo git remote add origin url path to add remote echo git remote
remove origin url path to remove remote echo git fetch origin to download latest changes echo git pull origin main to
fetch the data and merge it echo git push -u origin main to first time push the local data into remote

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/notes (main)
$ tail linux_day2.txt
linux day 2 notes
cat show full file content
head shows first lines
tail shows last lines
next goal is simple echo try to understand every concept in your own language first. echo for now i'll use a few git
commands echo git remote -v to check remote status echo git remote add origin url path to add remote echo git remote
remove origin url path to remove remote echo git fetch origin to download latest changes echo git pull origin main to
fetch the data and merge it echo git push -u origin main to first time push the local data into remote

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/notes (main)
$ head -n 3 linux_day2.txt
linux day 2 notes
cat show full file content
head shows first lines

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/notes (main)
$ tail -n 4 linux_day2.txt
cat show full file content
head shows first lines
tail shows last lines
next goal is simple echo try to understand every concept in your own language first. echo for now i'll use a few git
commands echo git remote -v to check remote status echo git remote add origin url path to add remote echo git remote
remove origin url path to remove remote echo git fetch origin to download latest changes echo git pull origin main to
fetch the data and merge it echo git push -u origin main to first time push the local data into remote

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/notes (main)
$ cat linux_day2.txt
linux day 2 notes
cat show full file content
head shows first lines
tail shows last lines
next goal is simple echo try to understand every concept in your own language first. echo for now i'll use a few git
commands echo git remote -v to check remote status echo git remote add origin url path to add remote echo git remote
remove origin url path to remove remote echo git fetch origin to download latest changes echo git pull origin main to
fetch the data and merge it echo git push -u origin main to first time push the local data into remote

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/notes (main)
$ |
```

Step 6: Permissions notes

Commands:

```
$ echo "r = read" > permissions.txt
```

Notes:

The echo command writes text into the file, and the cat command is used to display the complete contents of the file.

Screenshot:

```
Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/notes (main)
$ ls
linux_day2.txt  permission.txt  users.txt

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/notes (main)
$ echo "r stands read" > permission.txt

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/notes (main)
$ echo "w stands write" >> permission.txt

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/notes (main)
$ echo "x stands execute" >> permission.txt

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/notes (main)
$ echo "r+x stands read and execute" >> permission.txt

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/notes (main)
$ cat permission.txt
r stands read
w stands write
x stands execute
r+x stands read and execute
```

Step 7: User info

Commands:

```
$ whoami
$ id
$ groups
$ hostname
```

Notes:

users.txt file is populated using Linux commands to capture current user and system information. This demonstrates how command output can be stored in a file using the CLI.

Screenshot:

```
Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/notes (main)
$ echo "current user info:" > users.txt

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/notes (main)
$ whoami >> users.txt

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/notes (main)
$ id >> users.txt

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/notes (main)
$ groups >> users.txt
groups: cannot find name for group ID 197121

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/notes (main)
$ hostname >> users.txt

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/notes (main)
$ cat users.txt
current user info:
Sai
uid=197609(Sai) gid=197121 groups=197121
197121
Master
```

Step 8: Create backup.sh

Commands:

```
$ touch backup.sh
$ echo '#!/bin/bash' > backup.sh
$ echo 'echo "This is first shell script file"' >> backup.sh
```

Notes:

A shell script named **backup.sh** is created using the **touch command**. The script content is added using the **echo command**, verified with **cat**, and its permissions are checked using **ls -l** before execution.

Screenshot:

```
Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/notes (main)
$ cd ../scripts
Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/scripts (main)
$ touch backup.sh
Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/scripts (main)
$ echo '#!/bin/bash' > backup.sh
Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/scripts (main)
$ echo 'echo "This is first shell script file"' >> backup.sh
Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/scripts (main)
$ cat backup.sh
#!/bin/bash
echo "This is first shell script file"
Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/scripts (main)
$ ls -l backup.sh
-rwxr-xr-x 1 Sai 197121 51 Jul 11 06:26 backup.sh*
```

Step 9: Make executable

Commands:

```
$ ls -l backup.sh
$ chmod u+x backup.sh
$ ls -l backup.sh
```

Notes:

Before executing a shell script, execute permission must be granted using the **chmod u+x** command. Once the script becomes executable, it can be run using **./backup.sh**.

Screenshot:

```
Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/scripts (main)
$ ls -l backup.sh
-rwxr-xr-x 1 Sai 197121 51 Jul 11 06:26 backup.sh*
Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/scripts (main)
$ chmod u+x backup.sh
Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/scripts (main)
$ ls -l backup.sh
-rwxr-xr-x 1 Sai 197121 51 Jul 11 06:26 backup.sh*
```

Step 10: Run script

Commands:

```
$ ./backup.sh
```

Notes:

./backup.sh command executes the shell script, and the displayed output verifies that the script has been executed successfully.

Screenshot:

```
Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/scripts (main)
$ ./backup.sh
This is first shell script file
```

Step 11: Create bigfile

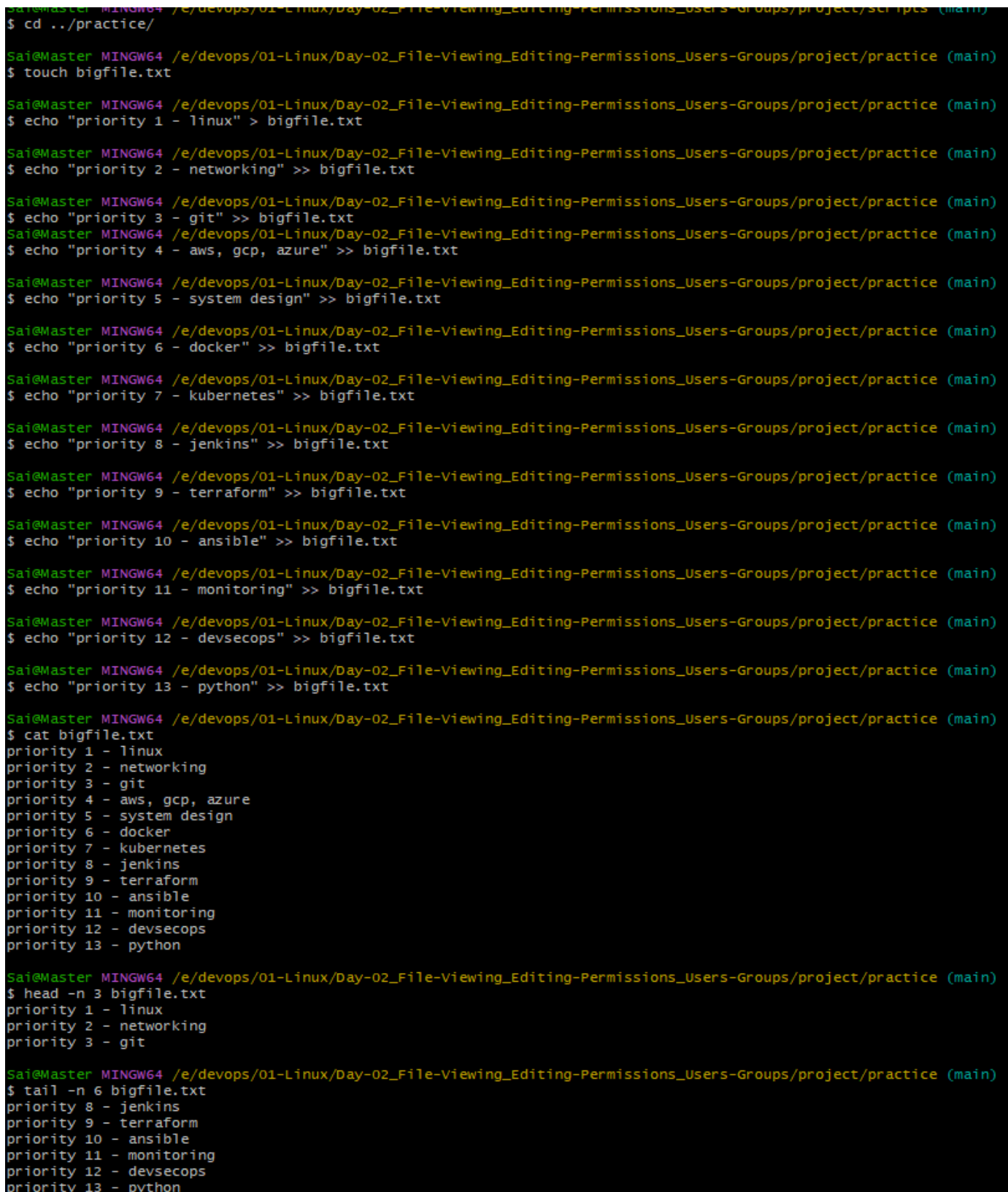
Commands:

```
$ touch bigfile.txt
$ Add 11 lines
$ cat/head/tail/less
```

Notes:

bigfile.txt file is created and filled using **echo** with the **>>** operator. The **cat**, **head**, and **tail** commands are then used to view the complete file, the first few lines, and the last few lines, respectively.

Screenshot:

A terminal window screenshot showing a series of commands and their outputs. The user is in a directory path: /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/practice. They create a file named bigfile.txt and then use the echo command with the >> operator to append 13 lines of text to it. The lines are: priority 1 - linux, priority 2 - networking, priority 3 - git, priority 4 - aws, gcp, azure, priority 5 - system design, priority 6 - docker, priority 7 - kubernetes, priority 8 - jenkins, priority 9 - terraform, priority 10 - ansible, priority 11 - monitoring, priority 12 - devsecops, and priority 13 - python. After creating the file, they use cat bigfile.txt to view the entire content, head -n 3 bigfile.txt to view the first three lines, and tail -n 6 bigfile.txt to view the last six lines.

```
Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/scripts (main)
$ cd ../practice/

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/practice (main)
$ touch bigfile.txt

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/practice (main)
$ echo "priority 1 - linux" > bigfile.txt

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/practice (main)
$ echo "priority 2 - networking" >> bigfile.txt

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/practice (main)
$ echo "priority 3 - git" >> bigfile.txt
Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/practice (main)
$ echo "priority 4 - aws, gcp, azure" >> bigfile.txt

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/practice (main)
$ echo "priority 5 - system design" >> bigfile.txt

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/practice (main)
$ echo "priority 6 - docker" >> bigfile.txt

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/practice (main)
$ echo "priority 7 - kubernetes" >> bigfile.txt

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/practice (main)
$ echo "priority 8 - jenkins" >> bigfile.txt

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/practice (main)
$ echo "priority 9 - terraform" >> bigfile.txt

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/practice (main)
$ echo "priority 10 - ansible" >> bigfile.txt

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/practice (main)
$ echo "priority 11 - monitoring" >> bigfile.txt

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/practice (main)
$ echo "priority 12 - devsecops" >> bigfile.txt

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/practice (main)
$ echo "priority 13 - python" >> bigfile.txt

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/practice (main)
$ cat bigfile.txt
priority 1 - linux
priority 2 - networking
priority 3 - git
priority 4 - aws, gcp, azure
priority 5 - system design
priority 6 - docker
priority 7 - kubernetes
priority 8 - jenkins
priority 9 - terraform
priority 10 - ansible
priority 11 - monitoring
priority 12 - devsecops
priority 13 - python

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/practice (main)
$ head -n 3 bigfile.txt
priority 1 - linux
priority 2 - networking
priority 3 - git

Sai@Master MINGW64 /e/devops/01-Linux/Day-02_File-Viewing_Editing-Permissions_Users-Groups/project/practice (main)
$ tail -n 6 bigfile.txt
priority 8 - jenkins
priority 9 - terraform
priority 10 - ansible
priority 11 - monitoring
priority 12 - devsecops
priority 13 - python
```